

Estructuras de Datos

Clase 6 – Lista Simplemente Enlazada



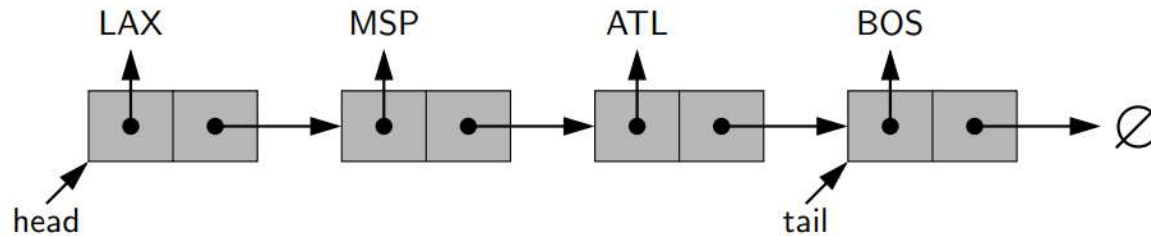
Dr. Sergio A. Gómez
<http://cs.uns.edu.ar/~sag>



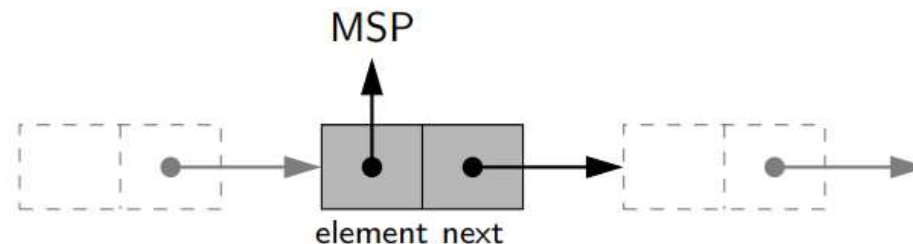
Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Repaso: Secuencias con nodos enlazados

- La secuencia [LAX, MSP, ATL, BOS] se representa con:



- Hay 2 campos *head* (para el primer elemento) y *tail* (para el último).
- Cada nodo conoce su elemento o rótulo (*element*) y el nodo siguiente en la secuencia (*next*) y se representa como:



- El símbolo $\emptyset$ indica que el siguiente es ninguno y, en Java, se representa con *null*.

Listas enlazadas (*Node lists* o *linked lists*)

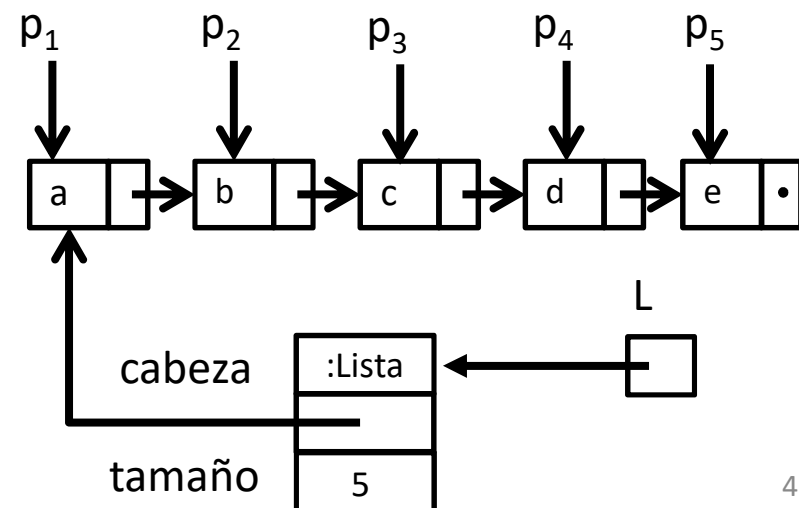
- Una lista L es una secuencia de n elementos $x_1, x_2, \dots, x_n$
- Representaremos a L como $[x_1, x_2, \dots, x_n]$
- Para implementarlas usaremos listas simple (o doblemente) enlazadas
- Usaremos la posición de nodo (en lugar de un índice como en arreglos) para referirnos a la “ubicación” de un elemento en la lista

Posiciones

- `element()`: Retorna el elemento ubicado en esta posición

```
public interface Position<E>
{
    // Retorna el valor del elemento ubicado en la posición
    public E element();
}
```

En una lista $L=[x_1, x_2, \dots, x_{n-1}, x_n]$, p_i es la posición de x_i para $i=1, \dots, n$. Esto se llama *posición directa*.

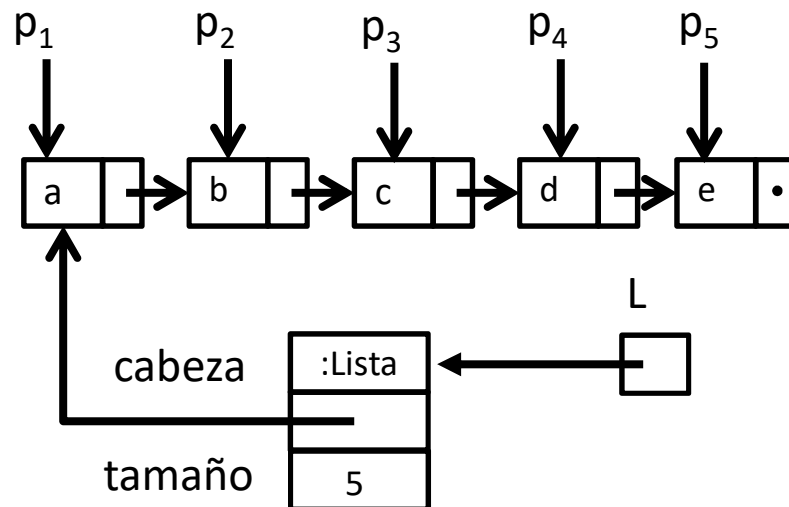


ADT Position List: (operaciones de iteración/recorrido)

- **first()**: retorna la posición del primer elemento de la lista; error si la lista está vacía
- **last()**: retorna la posición del último elemento de la lista; error si la lista está vacía
- **prev(p)**: retorna la posición del elemento que precede al elemento en la posición p; error si $p = \text{first}()$
- **next(p)**: retorna la posición del elemento que sigue al elemento en la posición p; error si $p = \text{last}()$

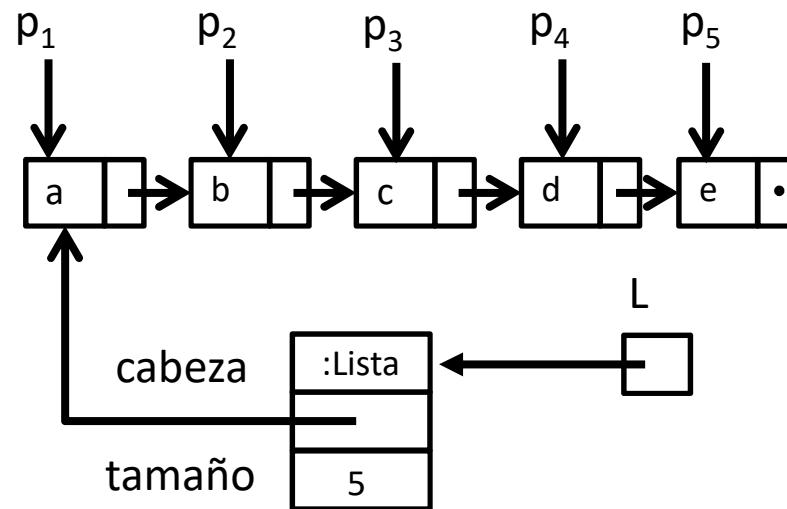
ADT Position List: Recorrido

- `first()`: retorna la posición del primer elemento de la lista; error si la lista está vacía
- Ejemplo:
 `L.first()` retorna p_1



ADT Position List: Recorrido

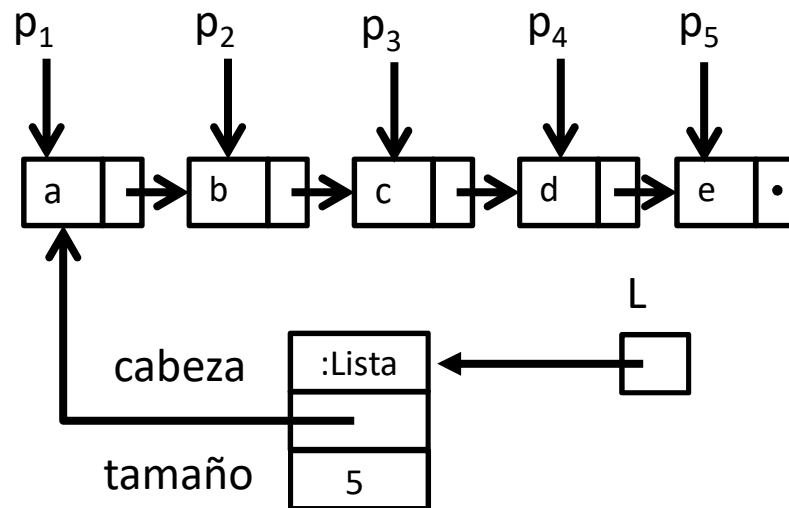
- `last()`: retorna la posición del último elemento de la lista; error si la lista está vacía
- Ejemplo:
 `L.last()` retorna p_5



L

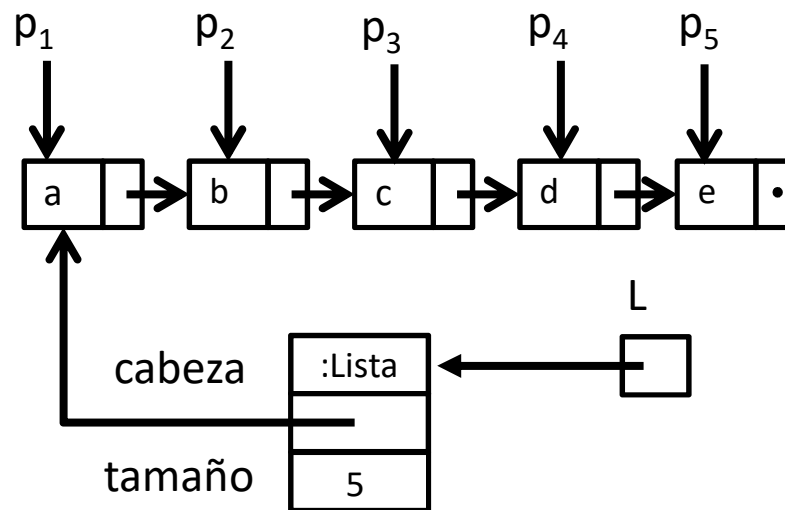
ADT Position List: Recorrido

- **prev(p)**: retorna la posición del elemento que precede al elemento en la posición p; error si $p = \text{first}()$
- Ejemplo:
 - L.prev(p_3) retorna p_2
 - L.prev(p_1) produce un error



ADT Position List: Recorrido

- **next(p)**: retorna la posición del elemento que sigue al elemento en la posición p; error si $p = \text{last}()$
- Ejemplo:
 - L.next(p_3) retorna p_4
 - L.next(p_5) produce un error

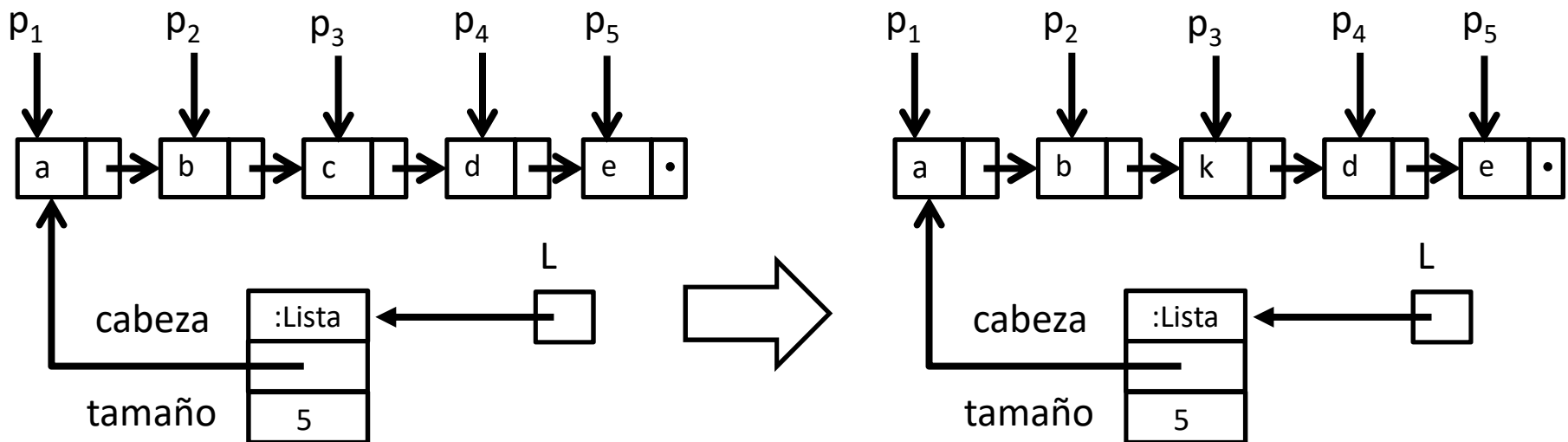


ADT Position List (métodos de actualización)

- `set( p, e )`: Reemplaza al elemento en la posición `p` con `e`, retornando el elemento que estaba antes en la posición `p`
- `addFirst(e)`: Inserta un nuevo elemento `e` como primer elemento
- `addLast(e)`: Inserta un nuevo elemento `e` como último elemento
- `addBefore(p, e)`: Inserta un nuevo elemento `e` antes de la posición `p`
- `addAfter(p, e)`: Inserta un nuevo elemento `e` luego de la posición `p`
- `remove(p)`: Elimina y retorna el elemento en la posición `p` invalidando la posición `p`.

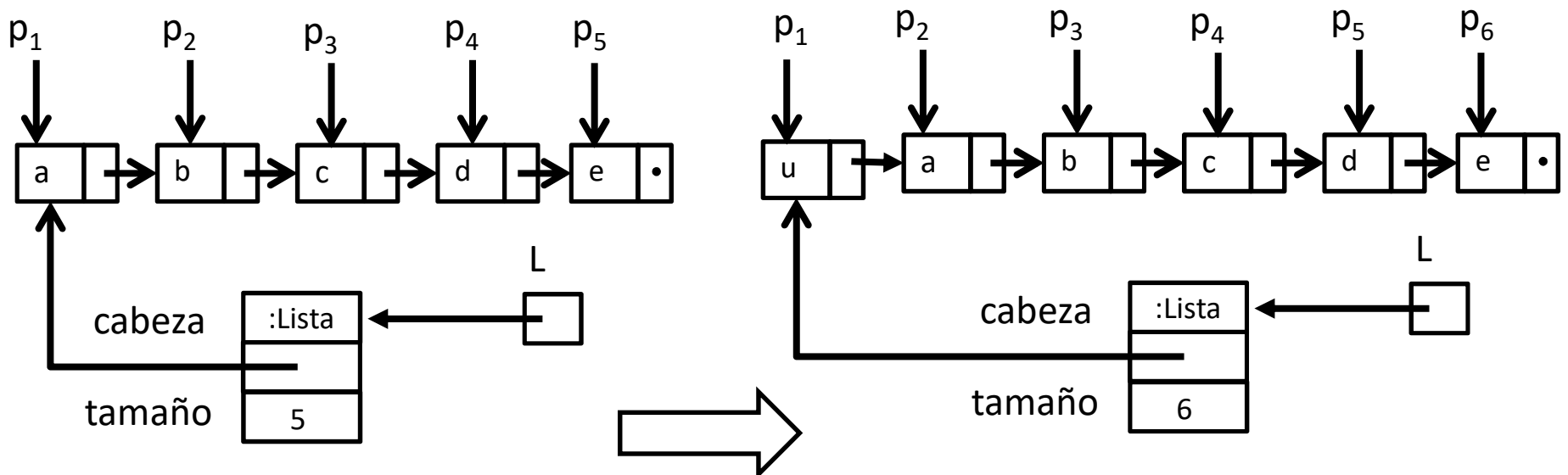
ADT Position List: Actualización

- `set( p, e )`: Reemplaza al elemento en la posición p con e, retornando el elemento que estaba antes en la posición p
- Ejemplo:
L.set(p_3 , 'k') retorna c y produce el siguiente efecto:



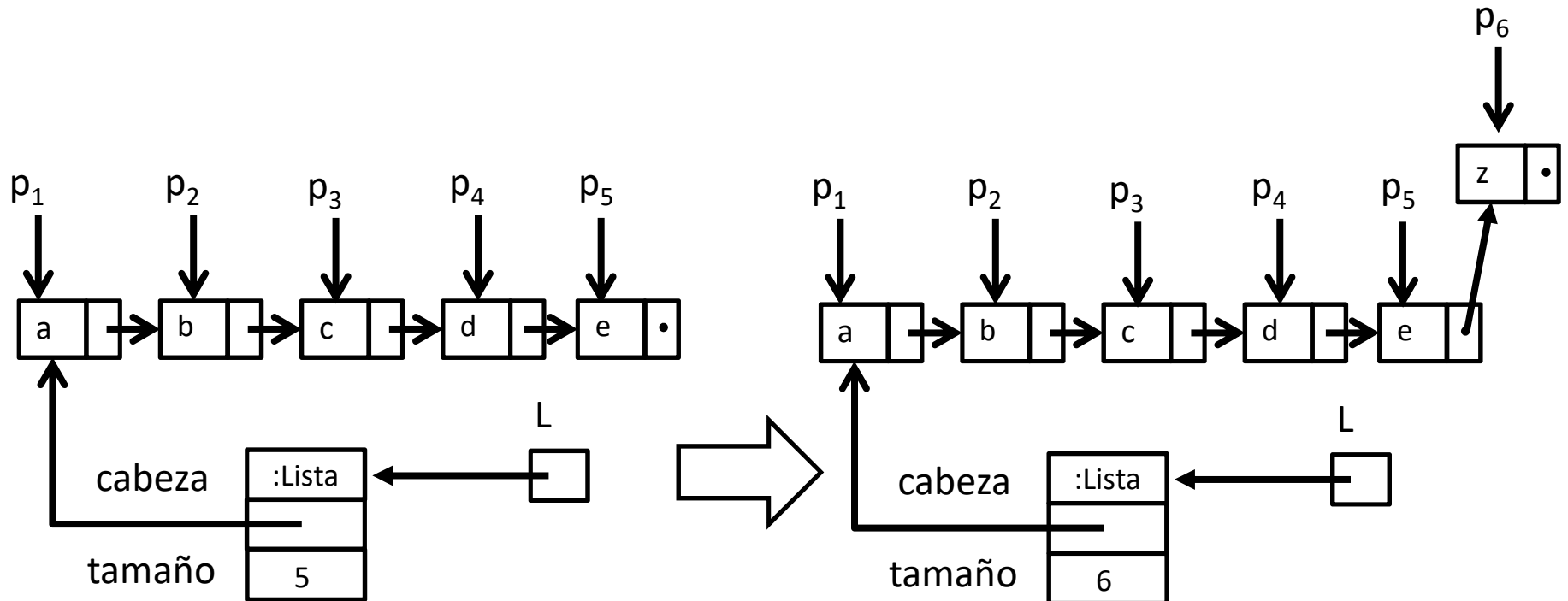
ADT Position List: Inserción

- **addFirst(e)**: Inserta un nuevo elemento e como primer elemento
- Ejemplo:
L.addFirst('u') produce el siguiente efecto:



ADT Position List: Inserción

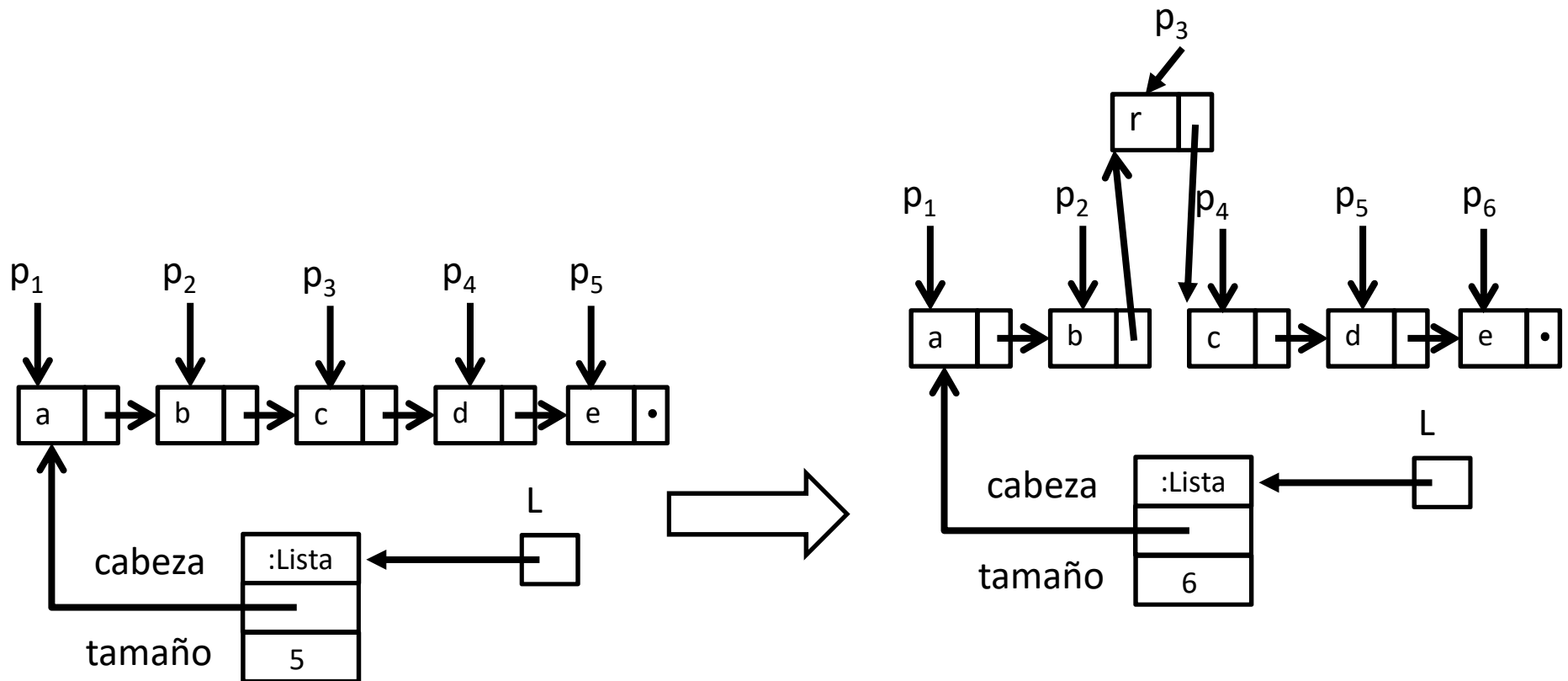
- **addLast(e)**: Inserta un nuevo elemento e como último elemento
- Ejemplo:
L.addLast('z') produce el siguiente efecto:



ADT Position List: Inserción

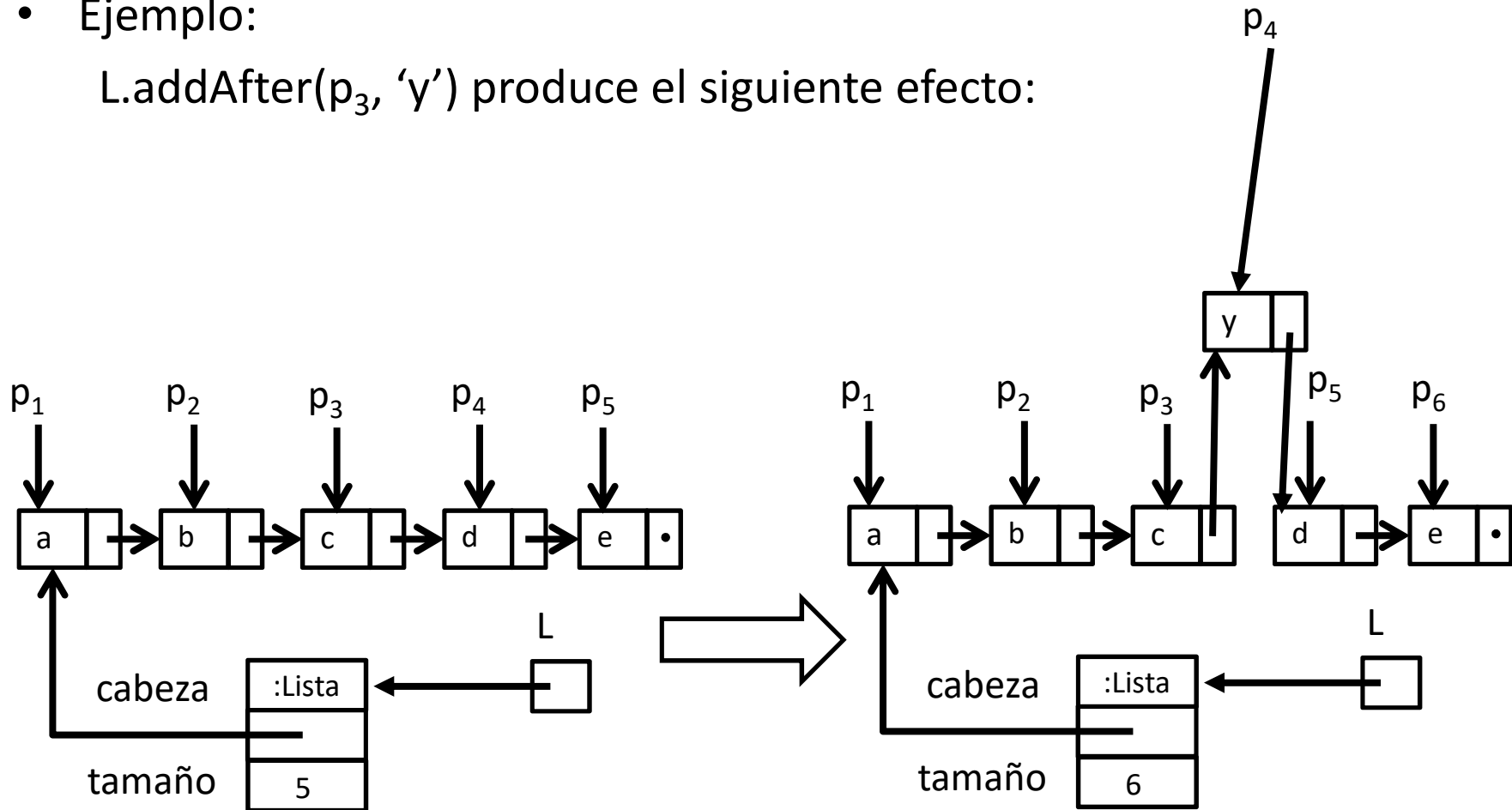
- `addBefore(p, e)`: Inserta un nuevo elemento `e` antes de la posición `p`
- Ejemplo:

`L.addBefore(p3, 'r')` produce el siguiente efecto:



ADT Position List: Inserción

- `addAfter(p, e)`: Inserta un nuevo elemento e luego de la posición p
- Ejemplo:
L.addAfter(p₃, 'y') produce el siguiente efecto:

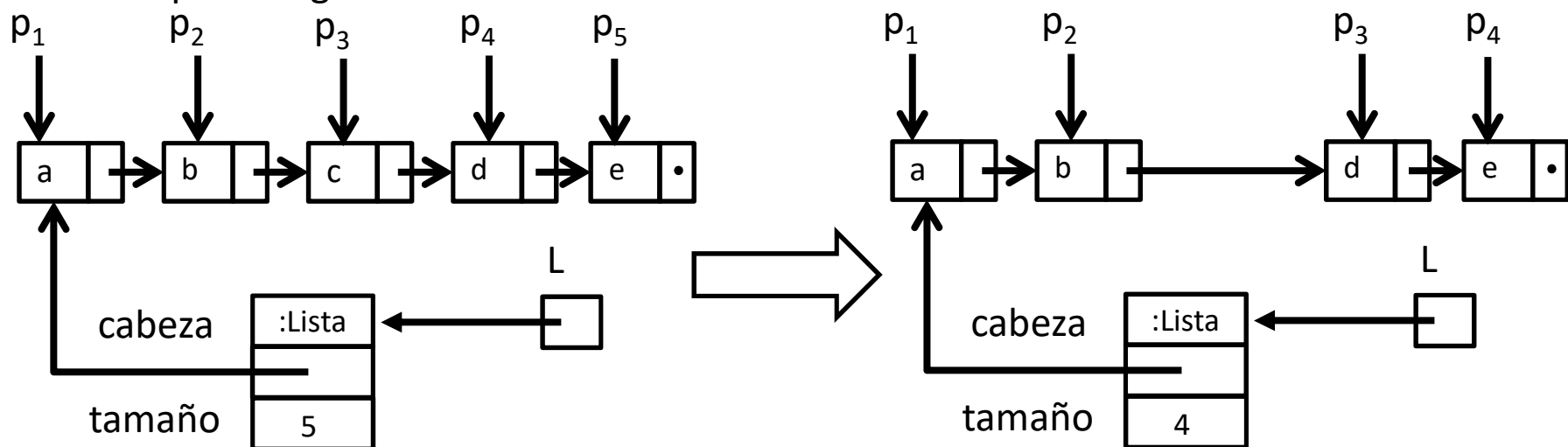


ADT Position List: Eliminación

- **remove(p)**: Elimina y retorna el elemento en la posición p invalidando la posición p.
- Ejemplo:

L.remove(p_3) produce el siguiente efecto:

Nota: En lenguajes con recolector de basura (como Java), la celda que contiene la c se deja “flotando” en el heap y en algún momento la reclama el recolector de basura. En lenguajes sin recolector (como C++) hay que liberar la celda por código.



Situaciones de error relacionadas a una posición p

1. $p = \text{null}$
 2. p fue eliminada previamente de la lista
 3. p es una posición de una lista diferente
 4. p es la primera posición de la lista e invocamos a $\text{prev}(p)$
 5. p es la última posición de la lista e invocamos a $\text{next}(p)$
- Nota: Por cuestiones de eficiencia, al implementar el TDA `PositionList` no validaremos (3) perdiendo un poco de robustez.
 - Nota: es posible validar (2) seteando el rótulo del nodo en `null` al eliminar la posición.
 - Nota: (3) podría ser validado recorriendo la lista para ver si la posición es válida o haciendo que el nodo conozca la lista que lo contiene. Nadie hace esto en ningún libro.

```
public interface PositionList<E> {  
    public int size();  
    public boolean isEmpty();
```

// Recorrido:

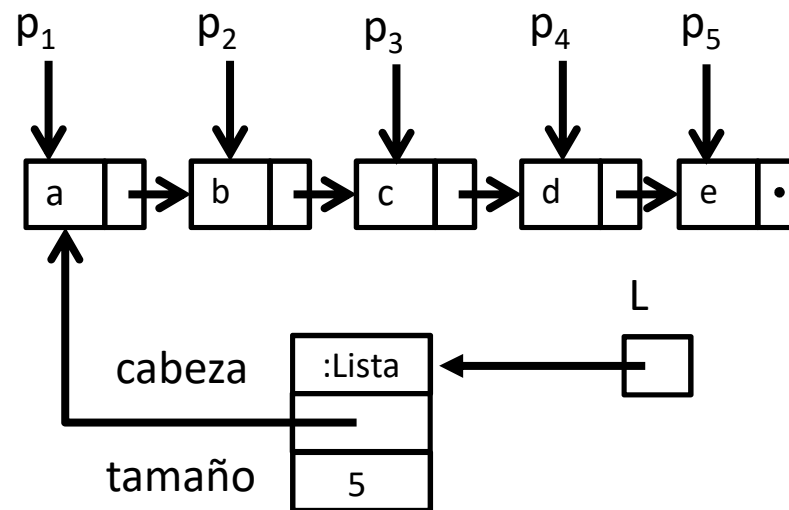
```
public Position<E> first(); // Error si lista vacía  
public Position<E> last(); // Error si lista vacía  
public Position<E> prev( Position<E> p ); // Error si p inválida o te caes de la lista.  
public Position<E> next( Position<E> p ); // Error si p inválida o te caes de la lista.
```

// Modificación:

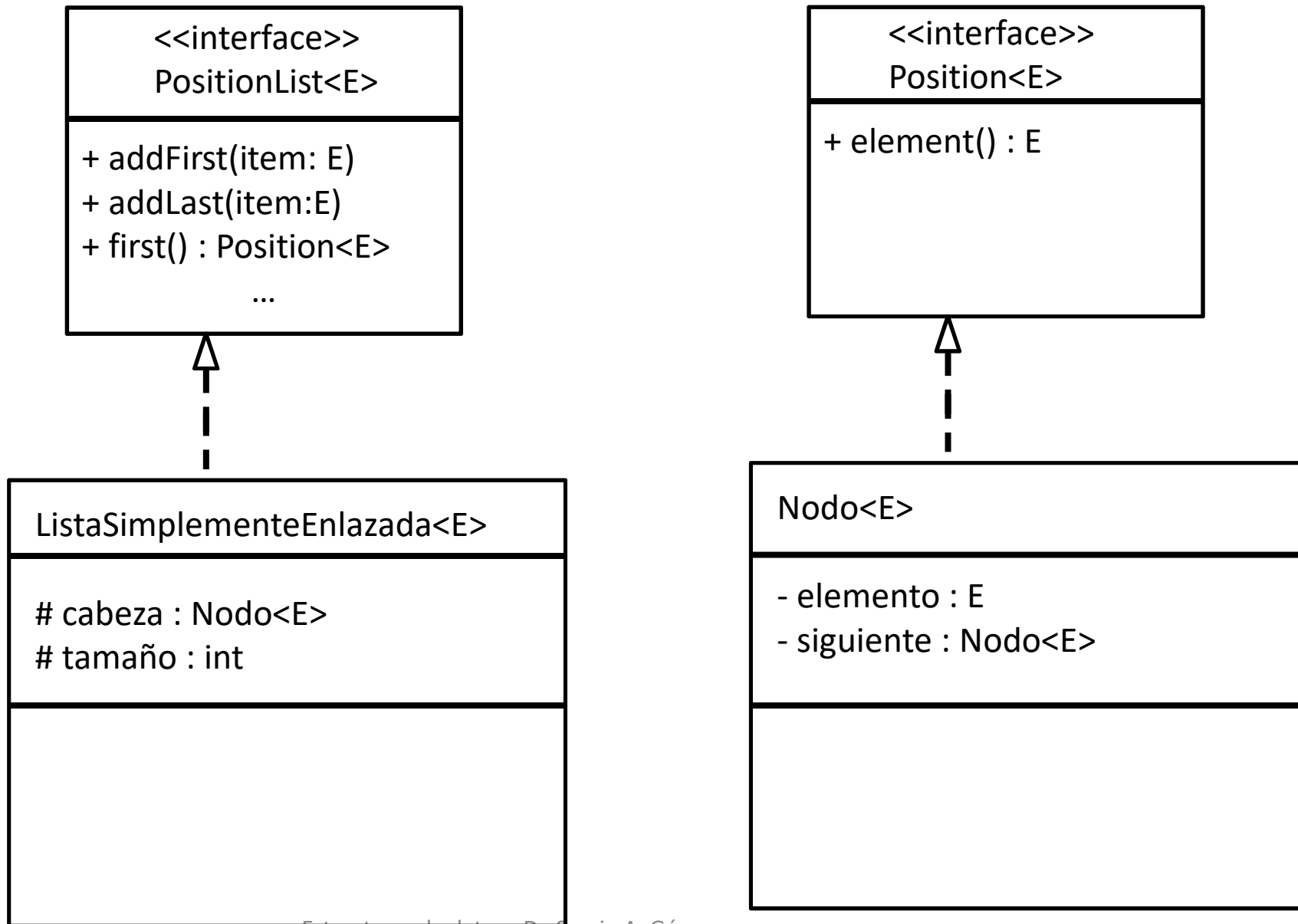
```
public void addFirst(E item);  
public void addLast(E item);  
public void addAfter( Position<E> p, E item ); // Error si p es inválida.  
public void addBefore( Position<E> p, E item); // Error si p es inválida.  
public E remove( Position<E> p ); // Error si p es inválida.  
public E set( Position<E> p, E item ); // Error si p es inválida.  
}
```

Implementación con listas simplemente enlazadas

- Lista simplemente enlazada: Una lista formada por nodos, donde cada nodo conoce un dato y la referencia al siguiente nodo
- La lista conoce la cabeza de la lista y su longitud
- Posición directa: La posición “p” de un nodo “n” es la referencia al nodo n.



Diseño UML de la implementación



Implementación con listas simplemente enlazadas y posición directa

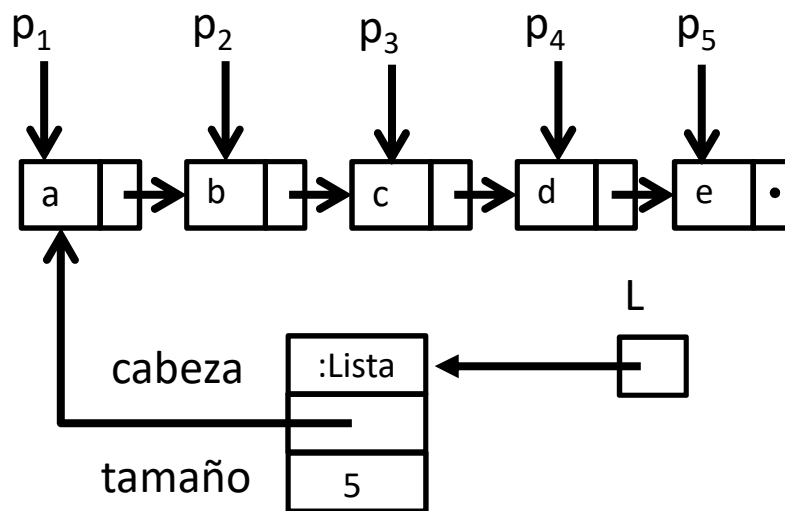
```
public class Nodo<E> implements Position<E> {  
    private E elemento;  
    private Nodo<E> siguiente;
```

... código ídem a clase Nodo<E> de Pilas y Colas enlazadas ...

// Nueva operación pedida por interfaz Position:

```
public E element() { return elemento; }
```

```
}
```



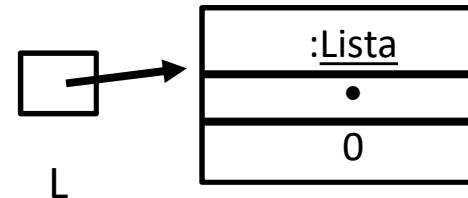
Implementación: Constructor, size e isEmpty

```
public class ListaSimplementeEnlazada<E> implements PositionList<E>
{
    protected Nodo<E> cabeza;
    protected int tamaño;

    public ListaSimplementeEnlazada() {
        cabeza = null;
        tamaño = 0;
    }

    public int size() { return tamaño; }
    public boolean isEmpty() { return cabeza == null; }

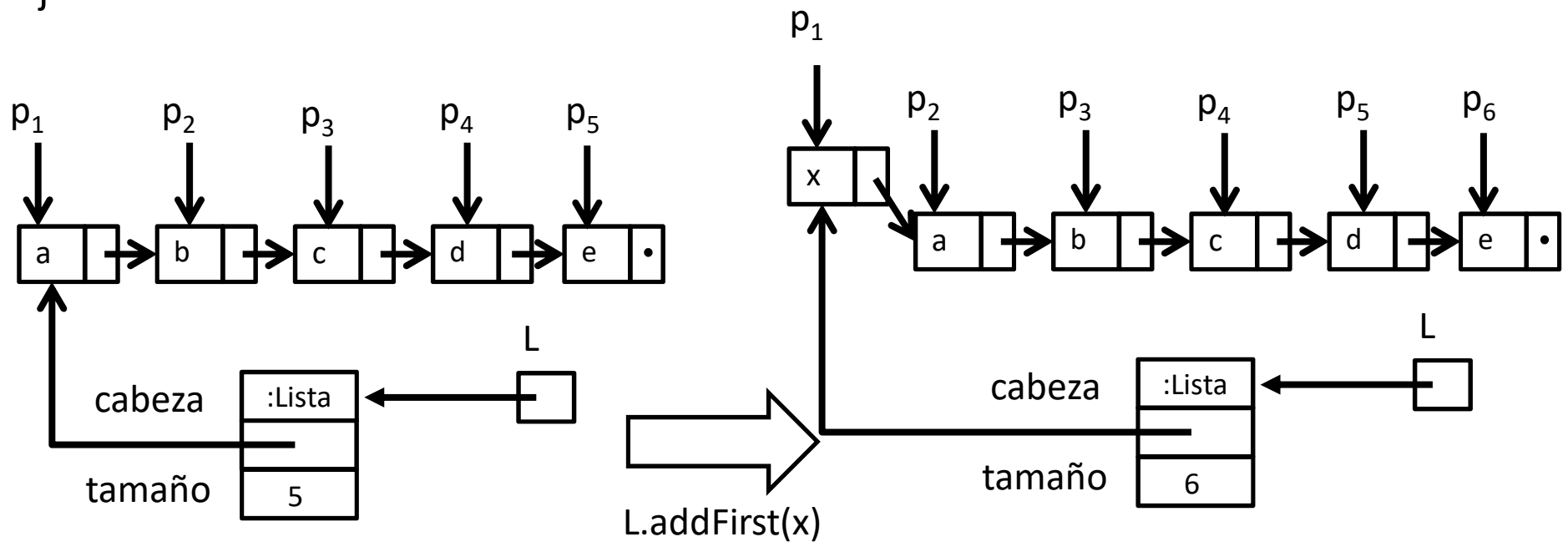
    // ... Operaciones de actualización y búsqueda aquí ...
}
```



Implementación: addFirst

El código es similar a *push* en pilas:

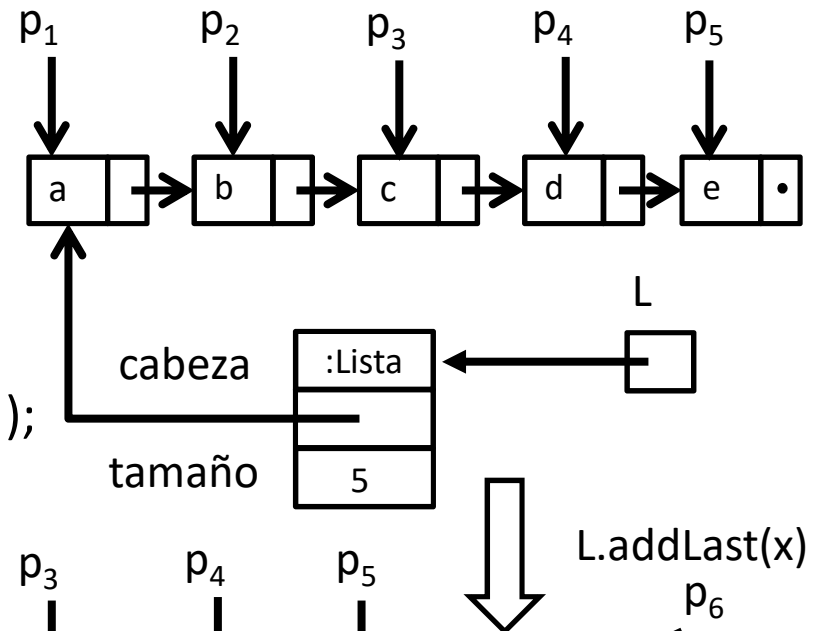
```
public void addFirst(E item ) {  
    cabeza = new Nodo<E>( item, cabeza );  
    tamaño++;  
}
```



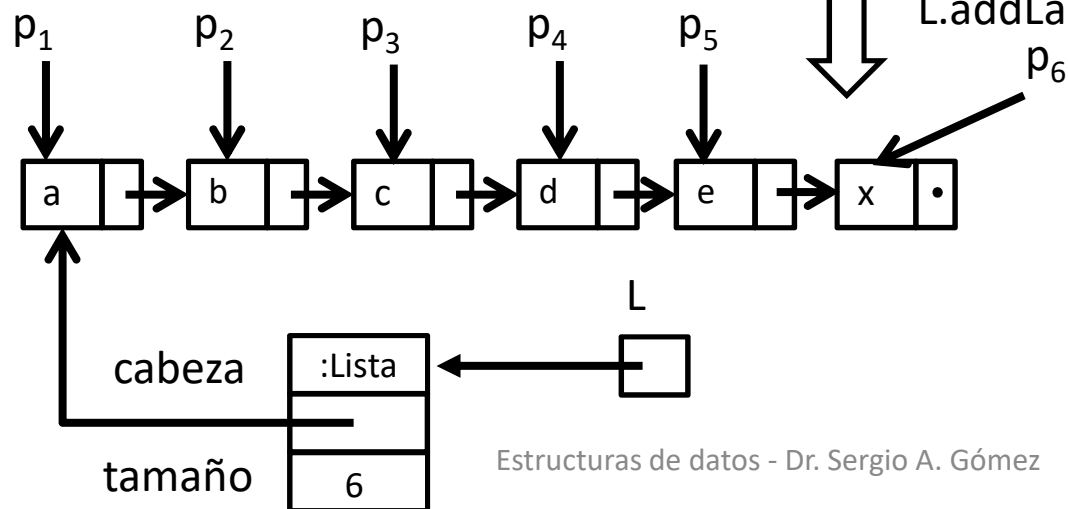
Implementación: addLast

La idea es la misma que en enqueue de colas excepto que ahora hay que recorrer los nodos hasta el último:

```
public void addLast( E item ) {  
    if( isEmpty() ) addFirst(item);  
    else {  
        Nodo<E> p = cabeza;  
        while( p.getSiguiente() != null )  
            p = p.getSiguiente();  
        p.setSiguiente( new Nodo<E>(item) );  
        tamaño++;  
    }  
}
```



Nota: El else se podría cambiar por `addAfter(last(),item)`.
Queda como ejercicio esta modificación.



Implementación: first y last

La operación first() retorna p1 cuando cuando la lista tiene elementos:
Lanza una EmptyListException cuando la lista está vacía.

```
public Position<E> first( ) {  
    if( isEmpty() )  
        throw new EmptyListException( "first():: Lista vacía" );  
    return cabeza;  
}
```

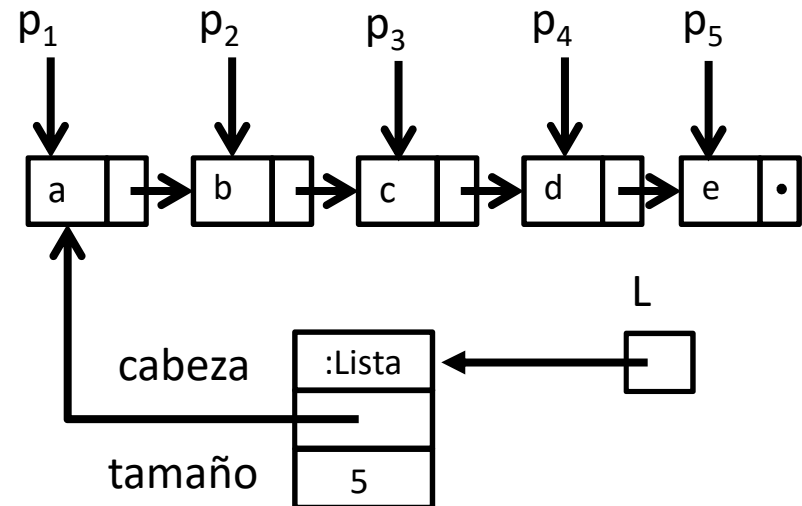
Cómputo de la última posición:

last() debe retornar p_n cuando la lista tiene $n > 0$ elementos.

Programarla usando el esquema del else de la operación addLast recorriendo la lista para encontrar p_n .

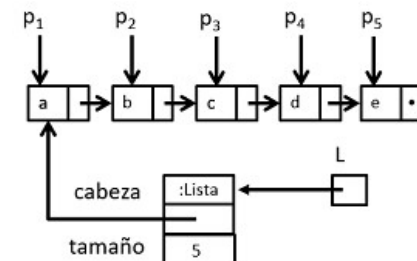
Lanzar excepción EmptyListException si la lista está vacía.

EmptyListException es una excepción del programador subclase de RuntimeException.



Implementación: next

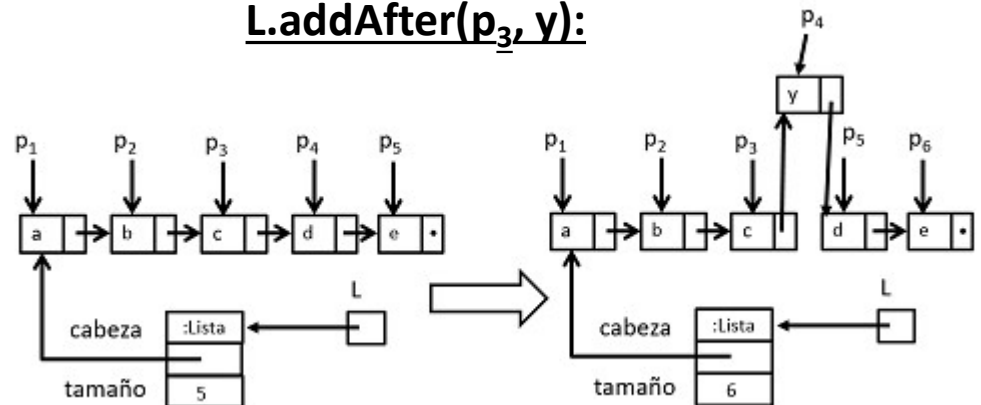
```
public Position<E> next( Position<E> p ) {  
    Nodo<E> n = checkPosition(p); // Se propaga una InvalidPositionException  
    if( n.getSiguiente() == null )  
        throw new BoundaryViolationException("Next:: Siguiente de último");  
    return n.getSiguiente();  
}  
  
private Nodo<E> checkPosition( Position<E> p ) {  
    try {  
        if( p == null ) throw new InvalidPositionException("Posición nula");  
        if (p.element() == null)  
            throw new InvalidPositionException("p eliminada previamente");  
        return (Nodo<E>) p; // Puede fallar si p es una posición que corresponde a un  
        // nodo de otro tipo de estructura de datos  
    } catch( ClassCastException e ) { // Vengo acá porque falló el casting a Nodo  
        throw new InvalidPositionException( "p no es un nodo de lista" );  
    }  
}
```



Implementación: addAfter

```
public void addAfter( Position<E> p , E item )  
    throws InvalidPositionException {  
    Nodo<E> n = checkPosition(p);    // Atención: Si esto falla, se propaga la excepción  
    Nodo<E> nuevo = new Nodo<E>(item);  
    nuevo.setSiguiente( n.getSiguiente() );  
    n.setSiguiente( nuevo );  
    tamaño++;  
}
```

L.addAfter(p₃, y):



¿Qué ocurre si p es una posición de otra lista?

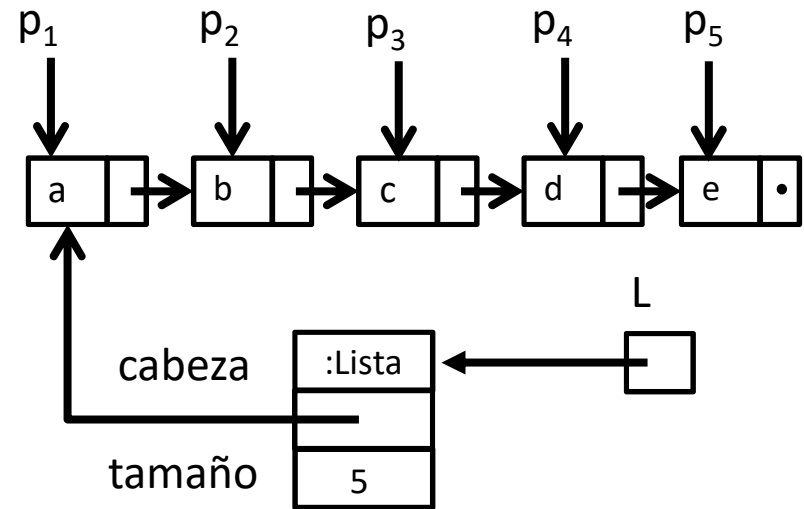
Implementación: prev

$L.\text{prev}(p_1) = \text{error}$

$L.\text{prev}(p_i) = p_{i-1}$, para $i = 2, \dots, n$

Hay que recorrer la lista desde el nodo cabeza hasta llegar al nodo aux cuyo siguiente es p.

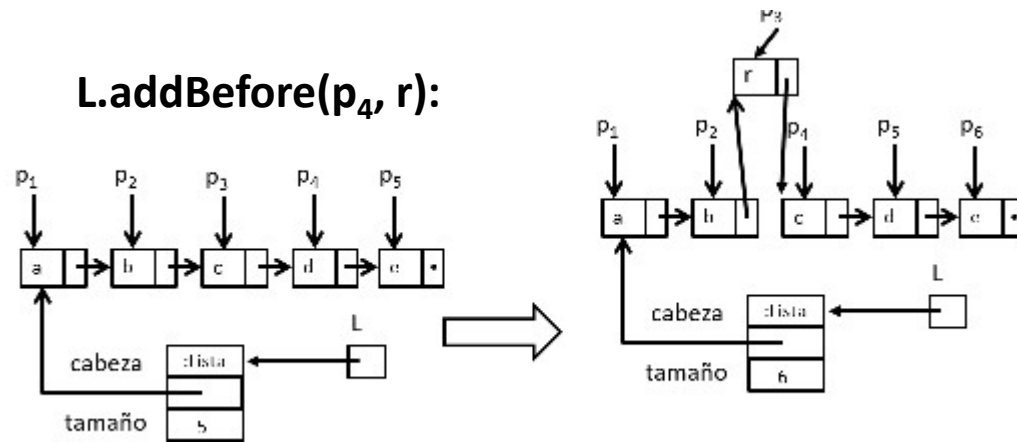
```
public Position<E> prev( Position<E> p ) {  
    ....  
    checkPosition(p);  
    if( p == first() ) throw new BoundaryViolationException("Posición primera" );  
    Nodo<E> aux = cabeza;  
    while( aux.getSiguiente() != p && aux.getSiguiente() != null )  
        aux = aux.getSiguiente();  
    if( aux.getSiguiente() == null ) // Si el cliente hizo las cosas bien, no ocurre  
        throw new InvalidPositionException("Posicion no pertenece a la lista" );  
    return aux;  
    ....  
}
```



Implementación: addBefore

Estrategia:

addBefore(p,e) debe buscar la posición previa a p e insertar a e luego de prev(p):



```
public void addBefore(Position<E> p, E item ) {
    ...
    checkPosition(p);
    if( p == first() ) addFirst(item);
    else {
        addAfter( prev(p), item );
    }
    ...
}
```

Implementación: remove

Remove(p_1) tiene que hacer que cabeza sea p_2

Remove(p_i) tiene que hacer que el siguiente de p_{i-1} sea p_{i+1} .

```
public E remove( Position<E> p ) {
```

```
    ...
```

```
    Nodo<E> n = checkPosition(p);
```

```
    ...
```

```
    if ( p == first() ) cabeza = n.getSiguiente();
```

```
    else checkPosition(prev(p)).setSiguiente( n.getSiguiente() );
```

```
    tamaño--;
```

```
    ...
```

```
    // Pongo los campos del nodo n en null para detectar en el futuro
```

```
    // que p es una posición inválida
```

```
    E aux = p.element();
```

```
    n.setElemento(null);
```

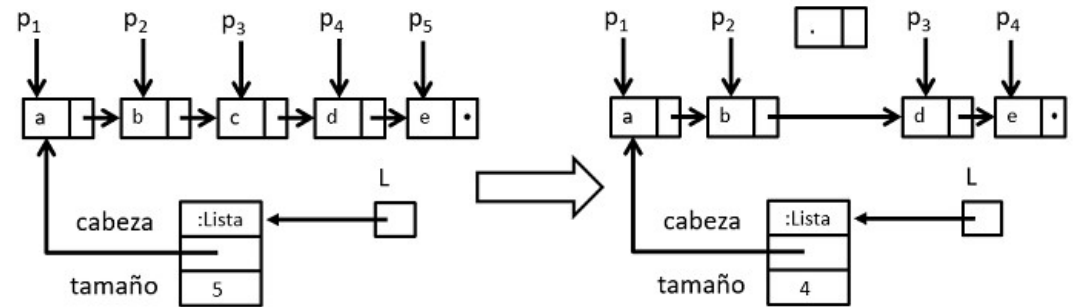
```
    n.setSiguiente(null);
```

```
    return aux;
```

```
    ...
```

```
}
```

L.remove(p3):



Discusión: Hackeo de la lista

Cuando la clase `Nodo` es pública es posible hackear la lista:

```
Position<String> p = l.first();
```

```
Nodo<String> n = (Nodo<String>) p;
```

Ahora el cliente usando la variable n puede acceder al estado interno de la lista l , lo que viola el principio de *abstracción de dato*.

Ahora técnicamente es possible escribir código como este:

```
n.setSiguiente( nodoDeOtraLista );
```

¡Esto se pone mal en los exámenes!

Discusión: Implementación segura

Haciendo anidada y privada a la clase `Nodo`, no es posible castear posiciones en nodos:

```
public class ListaSimplementeEnlazada<E> implements PositionList<E> {  
    protected Nodo<E> cabeza;  
    ...  
    private class Nodo<E> implements Position<E> { .... }  
    ....  
}
```

Ahora el casting a `Nodo` fuera de la clase `ListaSimplementeEnlazada` directamente no compila porque el identificador `Nodo` solo es referenciable dentro de la clase `ListaSimplementeEnlazada`.

Ejemplos

- Cargar una lista con valores
- Recorrido exhaustivo de una lista
- Recorrido no exhaustivo de una lista
- Insertar ordenadamente en una lista

Cargar una lista de enteros l con [1,2,3,4]

```
PositionList<Integer> l = new ListaSimplementeEnlazada<Integer>();  
l.addLast( 1 );  
l.addLast( 2 );  
l.addLast( 3 );  
l.addLast( 4 );
```

Cargar una lista de strings l2 con “Mama”, “Papa”, “Tio”:

```
PositionList<String> l2 = new ListaSimplementeEnlazada<String>();  
l2.addLast( “Tio” );  
l2.addFirst( “Papa” );  
l2.addFirst( “Mama” );
```

Recorrido exhaustivo de una lista para imprimirla

Algoritmo (à la GT): *Asume que la lista no está vacía*

Position<Character> p = L.first(), ultima = L.last();

```
while (p != null) {
    imprimir p.element()
    if (p == ultima)
        p = null;
    else
        p = L.next(p);
}
```

p toma los valores $p_1, p_2, p_3, p_4, p_5, \text{null}$
El if dentro del while parece una decisión pobre dentro del while. Ultima vale p_5

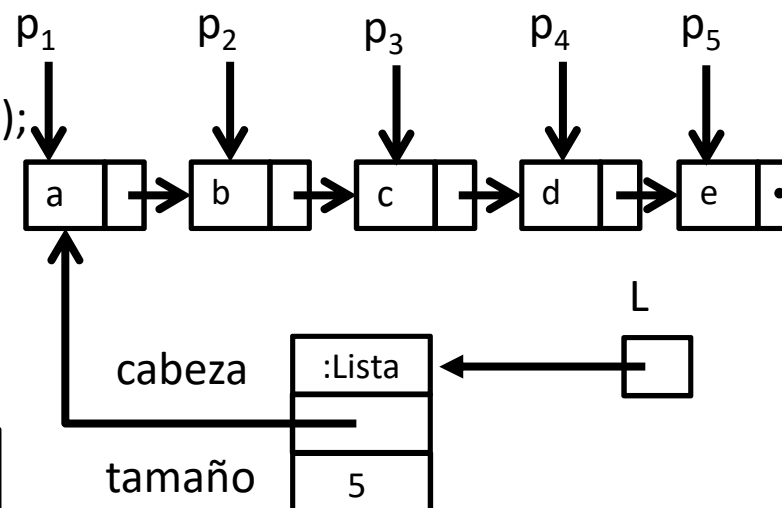
Algoritmo: *Asume que la lista no está vacía*

Position<Character> p = L.first(), ultima = L.last();

```
while (p != ultima) {
    imprimir p.element()
    p = L.next(p);
}
```

imprimir ultima.element();

p toma los valores p_1, p_2, p_3, p_4, p_5 .
El while es más elegante pero tengo que factorizar la acción a aplicar a p.element() para usarla nuevamente fuera del while.



Diapositiva 35

SAG1

Sergio A. Gómez; 24/4/2020

Recorrido no exhaustivo de una lista

Problema: Buscar un elemento x en una lista L no vacía:

```
boolean encuentre = false;
Position<Character> p = L.first(), ultima = L.last();
while ( p != null && !encontre ) {
    if (p.element().equals(x))
        encuentre = true;
    else if (p == ultima)
        p = null;
    else
        p = L.next(p);
}
```

Al salir p contiene la posición de x en L si y solo si encuentre es true.

Operador ternario ? :

El operador ternario ?: en Java es una forma concisa de escribir una estructura condicional if-else. Se utiliza para evaluar una expresión booleana y devolver uno de dos valores dependiendo del resultado.

Sintaxis: condición ? valor_si_verdadero : valor_si_falso;

- condición: una expresión que devuelve true o false.
- valor_si_verdadero: lo que se devuelve si la condición es true.
- valor_si_falso: lo que se devuelve si la condición es false.

Ejemplo:

```
int edad = 18;
```

```
String mensaje = (edad >= 18) ? "Es mayor de edad" : "Es menor de edad";
```

```
System.out.println(mensaje);
```

Imprime es mayor de edad.

Problema: Dada una lista l, insertar ítem en forma ordenada ascendente.

Estrategia: Recorrer la lista hasta encontrar el primer elemento mayor a ítem.

```
public void insertarOrdenado( PositionList<Integer> l, Integer item ) {  
    if( l.isEmpty() ) l.addFirst( item );  
    else {  
        Position<E> p = l.first(), ultima = l.last();  
        boolean encuentrePosicion = false;  
        while( p != null && !encuentrePosicion ) {  
            if ( item > p.element() ) p = p != ultima ? l.next(p) : null;  
            else encuentrePosicion = true;  
        }  
        if( encuentrePosicion ) l.addBefore( p, item );  
        else l.addAfter( ultima, item );  
    }  
}
```

Sumario

- Listas posicionales: ADT Position y PositionList
- Primera Implementación del ADT PositionList
 - La lista es una secuencia de nodos
 - La lista conoce al primer nodo y su longitud
 - Cada nodo conoce su elemento/rótulo y al nodo siguiente
 - Llegar a la última posición requiere recorrer toda la lista.
 - Determinar el nodo previo a un nodo determinado requiere recorrer la lista desde el comienzo.

Próxima clase

- Implementación con lista simplemente enlazada con enlace al primer y último elemento (como la ColaEnlazada) para que addLast no requiera recorrer.
- Listas circulares: la última celda conoce a la primera celda.
- Implementación con Lista doblemente enlazada con celdas centinelas al principio y al final para tener el nodo previo disponible.
- Iteradores para recorrer la lista en alto nivel.
- Pila y cola implementadas con listas

Bibliografía

- Goodrich & Tamassia, capítulo 6.

Informativo: Listas de Java: interfaz List

- Java brinda una versión de listas por medio de la interfaz `java.util.List<E>`.
- Las listas de Java son 0-basadas (el primer elemento es el 0 como en los arreglos).
- Implementa una secuencia $L=[x_0, x_1, x_2, \dots, x_{n-1}]$ de elementos de tipo genérico E.
- n es la longitud de L.
- La posición de cada elemento es un entero i entre 0 y n-1 tal que la posición de x_0 es 0, la de x_1 es 1, ..., la de x_{n-1} es n-1.
- Hay dos implementaciones de `List<E>`: `ArrayList<E>` y `LinkedList<E>`.

Listas de Java: ADT ArrayList

- `size()`: Retorna la cantidad de elementos de la lista S
- `isEmpty()`: Retorna verdadero si la lista S está vacía y falso en caso contrario
- `get(i)`: Retorna el elemento i-esimo de la lista S; ocurre un error si $i < 0$ o $i > \text{size}() - 1$
- `set(i,e)`: Reemplaza con e al elemento i-esimo; ocurre un error si $i < 0$ o $i > \text{size}() - 1$
- `add(i,e)`: Agrega un elemento e en posición i; ocurre un error si $i < 0$ o $i > \text{size}()$
- `remove(i)`: Elimina el elemento i-esimo de la lista S; ocurre un error si $i < 0$ o $i > \text{size}() - 1$

Ejemplo de ArrayList

Operación	Salida	S
add(0, 7)	-	[7]
add(0, 4)	-	[4, 7]
get(1)	7	[4, 7]
add(2, 2)	-	[4, 7, 2]
get(3)	error	[4, 7, 2]
remove(1)	7	[4, 2]
add(1, 5)	-	[4, 5, 2]
add(1, 3)	-	[4, 3, 5, 2]
add(4, 9)	-	[4, 3, 5, 2, 9]
get(2)	5	[4, 3, 5, 2, 9]
set(3, 8)	2	[4, 3, 5, 8, 9]

Listas nativas en Java

Clases `java.util.ArrayList<E>` y `LinkedList<E>` que implementan la interfaz `List<E>`

- `add(e)`: Inserta un elemento al final de la lista (append).
- `size()`: Retorna la cantidad de elementos de la lista L
- `isEmpty()`: Retorna verdadero si la lista L está vacía y falso en caso contrario
- `get(i)`: Retorna el elemento i-esimo de la lista L; ocurre un error si $i < 0$ o $i > \text{size}() - 1$
- `set(i,e)`: Reemplaza con e al elemento i-esimo; ocurre un error si $i < 0$ o $i > \text{size}() - 1$
- `add(i,e)`: Agrega un elemento e en posición i; ocurre un error si $i < 0$ o $i > \text{size}()$
- `remove(i)`: elimina el elemento i-esimo de la lista L; ocurre un error si $i < 0$ o $i > \text{size}() - 1$
- `clear()`: Elimina todos los elementos de la lista
- `toArray()`: retorna un array con los elementos de la lista en el mismo orden
- `indexOf(e)`: índice de la primera aparición de e en la lista
- `lastIndexOf(e)`: índice de la última aparición de e en la lista

```
import java.util.List;
import java.util.ArrayList;

public class EjemploListasJava {

    public static void main(String [] args) {

        // Creo una lista l de enteros:
        List<Integer> l = new ArrayList<Integer>();
        // con los elementos 3, 5, 7:
        l.add(3);
        l.add(5);
        l.add(7);

        System.out.println("L : " + l );
        // Imprime L : [3, 5, 7]

        // Imprimo la longitud:
        int largo = l.size();
        System.out.println("Largo: " + largo);
        // Imprime 3
    }
}
```

```
// Inserto un 8 delante de la posicion 2:  
l.add(2, 8);  
System.out.println("L : " + l );  
// Imprime L : [3, 5, 8, 7]
```

```
// Elimino el elemento de la posición 1:  
l.remove(1);  
System.out.println("L : " + l );  
// Imprime L : [3, 8, 7]
```

```
// Accedo a los elementos de a uno:  
for ( int i = 0; i < l.size(); i++ )  
    System.out.println( l.get(i) );
```

```
} // Fin main
```

```
} // Fin clase
```